

Simulation de la propagation d'un virus dans une population

Rapport de projet — Groupe 13

Système multi-agents · Algorithme A* · Communication réseau Python ↔ Java

Ejebli Adem

Application Java & rendu graphique

Ikdoumi Ilian

Moteur Python : agents, A*, mémoire, multithreading

Sabri Hicham

Communication réseau (sockets Python ↔ Java)

Projet réalisé dans le cadre de la Licence MIASHS — Université Paul Valéry Montpellier 3

Présentation du projet

Ce projet consiste à simuler la propagation d'un virus au sein d'une population d'agents autonomes se déplaçant dans une ville virtuelle. Chaque agent suit un parcours entre différents lieux (école, parc, travail, supermarché, etc.), se déplace de façon continue grâce à un algorithme A* adapté, et peut être contaminé en fonction de sa proximité avec d'autres agents infectés.

L'architecture repose sur deux programmes communiquant en temps réel : un moteur de simulation écrit en **Python** (gestion des agents, déplacements, infection, multithreading) et une application **Java** chargée de l'affichage graphique. Les deux échangent les positions et états des agents via des **sockets réseau**, tick par tick.

Composants techniques

Composant	Technologie	Rôle
Moteur de simulation	Python	Logique des agents, déplacement, infection, threads
Algorithme A*	Python	Déplacement continu par angles, évitement, mémoire
Application graphique	Java (Swing)	Affichage temps réel des agents et des lieux
Communication	Sockets TCP	Transfert positions + états Python → Java
Concurrence	Multithreading	Un thread par agent, synchronisation par cycle

Le rapport ci-après détaille l'avancement du projet semaine par semaine, en précisant pour chaque membre ses objectifs, le travail réalisé et les difficultés rencontrées.

SEMAINE 1

Mise en place de la structure et des premiers déplacements

Ejebli Adem

OBJECTIFS	TRAVAIL RÉALISÉ	PROBLÈMES RENCONTRÉS
• Créer la structure de base de l'application Java. • Créer les classes fondamentales : Agent, PanelSimulation et MainApp.	• Fenêtre principale (MainApp) avec intégration du panel de simulation. • Version initiale de Agent.java : nom, position, méthode draw() simple (cercle cyan). • Premier affichage statique.	• Mauvais centrage et mauvaise échelle de l'affichage.

Ikdoumi Ilian

OBJECTIFS	TRAVAIL RÉALISÉ	PROBLÈMES RENCONTRÉS
• Modifier la fonction A* qui fonctionnait dans un monde non continu. • Remplacer le déplacement case par case par un déplacement par directions. • Permettre aux agents de se déplacer en diagonale.	• La fonction A* a été transformée pour que les agents ne bougent plus de case en case. • Les agents se déplacent désormais par directions : tout droit, diagonales, etc. • Un test simple a confirmé le bon fonctionnement sur un chemin.	• Difficultés liées au passage d'un déplacement discret à un déplacement par angles dans un monde continu.

Sabri Hicham

OBJECTIFS	TRAVAIL RÉALISÉ	PROBLÈMES RENCONTRÉS
• Comprendre le fonctionnement des sockets pour la communication réseau. • Commencer la création du serveur Python qui enverra les données vers Java.	• Étude du principe des sockets et de leur usage en Python. • Mise en place du début du serveur Python, base de la future communication. • Définition de la structure d'envoi des informations des agents vers Java.	• Le serveur Java n'est pas encore créé, ce qui empêche de tester la communication complète. • L'envoi des données vers Java sera finalisé la semaine suivante.

SEMAINE 2

Communication réseau et premiers agents en mouvement

Ejebli Adem

OBJECTIFS	TRAVAIL RÉALISÉ	PROBLÈMES RENCONTRÉS
- Recevoir les données envoyées par Python. - Afficher un agent en temps réel.	- Mise en place de NetworkThread.java. - Lecture du protocole Python : $\text{nom} + x + y$. - Ajout de <code>updateAgents()</code> dans <code>PanelSimulation</code>. - Affichage des agents en mouvement.	Aucun problème notable.

Ikdoumi Ilian

OBJECTIFS	TRAVAIL RÉALISÉ	PROBLÈMES RENCONTRÉS
- Améliorer la gestion du déplacement continu dans A*. - Mettre en place une structure pour créer plusieurs agents distincts.	- Modification d'A* pour calculer le meilleur angle vers le lieu cible. - Création de la classe Agent (vitesse, parcours, nom). - Chaque agent se déplace désormais en continu selon sa vitesse et la direction choisie par A*.	- L'agent ne possède pas encore de mémoire des zones explorées. - Aucun champ de vision n'est encore implémenté, ce qui limite la détection.

Sabri Hicham

OBJECTIFS	TRAVAIL RÉALISÉ	PROBLÈMES RENCONTRÉS
- Tester la communication réseau entre le serveur Python et le client Java. - Vérifier que le serveur Python est prêt à recevoir la connexion.	- Mise en place et lancement du serveur Python, qui fonctionne correctement. - Tentative de connexion depuis Java pour vérifier la communication.	- Le client Java n'arrivait pas à se connecter au serveur Python. - La communication n'a pas encore pu être établie ; à corriger ensuite.

SEMAINE 3

Affichage des lieux, vision et mémoire des agents

Ejebli Adem

OBJECTIFS	TRAVAIL RÉALISÉ	PROBLÈMES RENCONTRÉS
Afficher les lieux envoyés par Python.	- Décodage des lieux Python : nom + coordonnées. - Affichage des lieux à l'écran.	Affichage de manière discontinue.

Ikdoumi Ilian

OBJECTIFS	TRAVAIL RÉALISÉ	PROBLÈMES RENCONTRÉS
- Ajouter un système de vision pour détecter l'environnement. - Permettre à chaque agent de mémoriser les lieux découverts.	- Ajout d'un rayon de vision pour analyser l'environnement de l'agent. - Création de l'attribut lieuxDécouverts (liste des lieux repérés, vide au départ). - À chaque appel d'updateVision(), les lieux visibles sont ajoutés automatiquement. - L'agent découvre ainsi progressivement la carte et reconnaît les lieux déjà vus.	- La mémoire est insuffisante : elle ne retient que les lieux découverts. - Elle devrait être une grille complète mémorisant lieux, zones vides et zones explorées.

Sabri Hicham

OBJECTIFS	TRAVAIL RÉALISÉ	PROBLÈMES RENCONTRÉS
- Finaliser la connexion entre serveur Python et client Java. - Vérifier que les données Python sont bien reçues côté Java.	- La connexion Python ↔ Java fonctionne désormais correctement. - Les agents apparaissent côté Java, positions reçues tick par tick. - La communication réseau est opérationnelle.	- L'envoi depuis Python reste discontinu, pas encore fluide. - Cela provoque un affichage irrégulier du déplacement ; priorité de la semaine suivante.

Ejebli Adem

OBJECTIFS	TRAVAIL RÉALISÉ	PROBLÈMES RENCONTRÉS
• Préparer l'affichage avancé (sans encore l'infection). • Rendre le code plus stable avant les grosses fonctionnalités.	• Amélioration du rendu graphique. • Mise au propre du PanelSimulation. • Ajout de zones jaune et rouge autour des lieux pour bien les délimiter.	• Avec plusieurs agents à l'écran, l'affichage devient instable et les agents clignotent.

Ikdoumi Ilian

OBJECTIFS	TRAVAIL RÉALISÉ	PROBLÈMES RENCONTRÉS
• Revoir entièrement le système de mémoire des agents. • Intégrer cette mémoire dans A* pour améliorer l'exploration et l'évitement.	• La mémoire devient une matrice largeur × longueur de la carte. • Chaque case : 0 = inconnue, 1 = connue et sûre, 2 = présence d'un lieu. • Tout lieu aperçu est immédiatement enregistré dans la mémoire de l'agent. • A* utilise cette matrice pour éviter les lieux (2) et anticiper ses trajets.	• L'agent connaît l'emplacement des lieux à l'avance, ce qui fausse son exploration. • Il devra chercher son premier lieu tant qu'il ne l'a pas découvert lui-même.

Sabri Hicham

OBJECTIFS	TRAVAIL RÉALISÉ	PROBLÈMES RENCONTRÉS
• Améliorer la communication Python ↔ Java. • Obtenir un envoi continu pour un affichage fluide.	• Le problème d'envoi discontinu depuis Python a été résolu. • Les données sont désormais envoyées en continu, sans interruption. • Côté Java, le déplacement des agents est maintenant fluide.	• Avec plus d'un agent, les agents clignotent. • Cause : Java n'affiche pas tous les agents en même temps lors du rendu. • À résoudre : gérer l'affichage simultané de plusieurs agents.

Ejebli Adem

OBJECTIFS	TRAVAIL RÉALISÉ	PROBLÈMES RENCONTRÉS
• Ajouter toute la logique sanitaire : infection, guérison, immunité, propagation. • Colorer les agents selon leur état sanitaire. • Ajouter les statistiques en direct.	• Ajout de l'état (state) au protocole : 0 = sain, 1 = infecté, 2 = immunisé. • NetworkThread mis à jour pour lire l'état ; couleurs : cyan = sain, rouge = infecté, vert = guéri. • Ajout d'un compteur de guérison basé sur les lieux parcourus. • Comptage automatique des sains / infectés / immunisés, affiché dans un JLabel de MainApp.	<i>Aucun problème notable.</i>

Ikdoumi Ilian

OBJECTIFS	TRAVAIL RÉALISÉ	PROBLÈMES RENCONTRÉS
• Mettre en place une gestion correcte des threads (un par agent). • Synchroniser les parcours des agents entre chaque cycle. • Résoudre le problème des agents connaissant leurs lieux à l'avance.	• Système multithreadé : un thread par agent, parcours indépendants, attente mutuelle entre cycles. • Les agents ne connaissent plus leurs lieux au départ : ils explorent d'abord aléatoirement. • Dès qu'un lieu est découvert par la vision, A* classique est utilisé pour s'y rendre. • Recherche structurée par découpage de la carte en 4 cadrans (SO, SE, NO, NE) explorés successivement.	<i>Aucun problème notable.</i>

Sabri Hicham

OBJECTIFS	TRAVAIL RÉALISÉ	PROBLÈMES RENCONTRÉS
• Corriger le clignotement de l'affichage des agents. • Intégrer les fonctionnalités d'infection et de collision.	• Le clignotement est entièrement résolu : tous les agents sont affichés en même temps. • Ajout d'un contrôle dynamique de la vitesse d'affichage en Java. • Infection calculée selon la distance : en zone à risque, probabilité de contamination. • Système de collision par distance : les agents trop proches s'évitent automatiquement.	<i>Aucun problème notable.</i>